MINI PROJECT REPORT

SMART KEYPAD LOCK USING DETERMINISTIC FINITE AUTOMATA

Submitted by:

Easala Jagadeesh

Roll No.: 2411021061183

Email ID: easala.jagadeesh.s71@kalvium.community, jeasalaBTECH24@ced.alliance.edu.in

Squad: 71

In partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Vaibhav Giri

Associate Professor, ASAC

Alliance School of Advanced Computing

Alliance University, Bengaluru

October-2026



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the report on **Formal Languages and Automata Theory** is submitted by **Easala Jagadeesh** [Uni. Reg. No. 2411021061183] in partial fulfilment for the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27. This report embodies the results of original work and studies conducted by the student and the contents do not form the basis for the award of any other degree to the candidate or anybody else.

- - - - - - - - - - - - - - - - - - - -

Vaibhav Giri

Associate Professor



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the report titled **SMART KEYPAD LOCK USING DETERMINISTIC FINITE AUTOMATA**, submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Vaibhav Giri, Associate Professor.

I further confirm that all views, findings, and conclusions expressed herein are the result of sincere effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Easala Jagadeesh	**2411021061183**	
Name of the Student	University Registration Number	Signature

Acknowledgement

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this project titled **SMART KEYPAD LOCK USING DETERMINISTIC FINITE AUTOMATA**.

First and foremost, I extend my sincere thanks to my mentor and guide, Vaibhav Giri, for the unwavering support, insightful feedback, and encouragement throughout the development of this system. The guidance received in the subject Formal Languages and Automata Theory provided the perfect foundation for this work.

I am deeply grateful to my faculty members and peers whose guidance and technical inputs helped refine the project. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project allowed me to turn the definition of a deterministic finite automaton into a working device, to construct a DFA systematically from a pattern, and to see how a small error in an informal design can be found and fixed with automata theory.

Lastly, I would like to thank everyone who supported me directly or indirectly during this journey. Your encouragement and belief in my work made this accomplishment possible.

Thank you.

Easala Jagadeesh

TABLE OF CONTENTS

Acknowledgement **4**

Abstract **6**

Chapter I- Introduction **7**

Problem Statement 7

Theory 7

Chapter II - System Configuration **9**

Algorithm 9

Implementation 10

Source Code 10

Chapter III - Results **13**

Sample inputs and outputs 13

The DFA for the code 1213 14

DFA lock against naive lock 14

How often the naive lock fails 15

Exhaustive verification 16

Chapter IV - Test Cases **17**

Construction of the DFA 17

Simulation of key sequences 17

Invalid input and performance 18

Analysis of the test results 18

Chapter V - Conclusion **20**

Abstract

Digital keypad locks are used on doors, lockers and safes. The lock must open as soon as the last keys pressed match the secret code, even if the user pressed wrong keys first. A simple design counts how many correct keys have been pressed in a row and starts again from zero on a wrong key. This design has a hidden flaw: with the code 1213, the key sequence 1 2 1 2 1 3 ends with the code, yet the lock stays shut.

This report models the lock as a deterministic finite automaton $M = (Q, \Sigma, \delta, q_0, F)$ over the ten digit keys. State i means that the last i keys match the first i digits of the code, and the transition function is constructed so that, after a wrong key, the automaton falls back to the longest part of the code that is still matched instead of restarting.

For the code 1213 the DFA has five states and opens correctly on 121213, where the naive lock fails. An exhaustive check of 136,710 key sequences over seven different codes confirmed that the DFA opens exactly when the code has been entered, while the naive lock missed up to 13.9% of the sequences that should open it.

The program takes any secret code and key sequence as input and simulates both locks key by key, showing each DFA move, what is matched, and where the naive lock forgets a partial match. 17 of 17 test cases passed, covering the construction, the simulation, invalid input and 1,000,002 key presses.

Chapter I- Introduction

Formal language theory studies sets of strings and the abstract machines that recognise them. Regular languages are the simplest class and are recognised by finite automata, machines with a fixed, finite number of states that read their input one symbol at a time. Because they need no memory beyond the current state, finite automata are ideal for small embedded devices.

A keypad lock is a natural finite automaton: its input is the stream of key presses and its only job is to decide, after every key, whether to open. This report designs that automaton systematically and compares it with an informal design.

Problem Statement

The lock must open whenever the trailing keys equal the secret code of length k, whatever keys were pressed before. The informal 'naive' design keeps a count s of correct keys: on the expected key it increases s, on any other key it resets s to 0 (or to 1 if the key equals the first digit of the code), and it opens when s = k. With the code 1213 and the keys 121213, the first three keys give s = 3; the fourth key, 2, is not the expected 3, so s is reset to 0, and the lock never opens, although the last four keys are 1213. The problems addressed are:

- to describe the language of key sequences that must open the lock;
- to construct a DFA for any secret code that never loses a partial match;
- to demonstrate the naive design's failure and measure how often it occurs; and
- to verify the DFA exhaustively against the definition.

Theory

Deterministic finite automata

A DFA is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states, Σ is the input alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the transition function, q_0 is the start state and F is the set of accepting states. The DFA reads a string one symbol at a time, follows exactly one transition per symbol and accepts if it ends in an accepting state.

Component	Smart lock DFA for a code P of length k
Q	{0, 1, ..., k}: state i means the last i keys equal the first i digits of P
Σ	{0, 1, 2, ..., 9}, the ten keys
q_0	0 (nothing matched yet)
F	{k}: the whole code has been entered, so the lock opens
$\delta(i, x)$	length of the longest prefix of P that is a suffix of (first i digits of P) + x

Table 1: The smart lock as a 5-tuple

The language of the lock

The lock must open for every key sequence that contains the code P: the language is $L = \Sigma^* P \Sigma^*$, all strings over the keys that contain P as a substring. L is described by a regular expression, so by Kleene's theorem it is regular and a DFA for it exists.

Constructing the transition function

When the DFA is in state i, the last i keys are exactly the first i digits of P. After a new key x, the string (P[0..i−1] + x) is known, and the new state is the length of its longest suffix that is also a prefix of P. If x is the expected digit this is simply i + 1. If x is wrong, the DFA does not throw away everything: it falls back to the longest shorter match. For P = 1213 in state 3 ('121'), the key 2 gives '1212', whose longest suffix that is a prefix of P is '12', so $\delta(3, 2) = 2$, not 0. This is the transition the naive design gets wrong. The same idea is used by the Knuth-Morris-Pratt string-matching algorithm.

Cost

Building the table computes k + 1 rows of 10 entries, each by comparing at most k suffixes of length at most k, so construction takes $O(10 \cdot k^3)$ time, which is tiny for a passcode. Running the DFA takes exactly one table lookup per key, so a sequence of n keys is processed in $\Theta(n)$ time with constant memory (just the current state).

Chapter II - System Configuration

Smart Lock DFA is written in Python 3 without external libraries. It has a terminal program (with a demonstration mode, a single-sequence mode and an interactive keypad mode) and a web interface with an on-screen keypad, both locks side by side, the state graph and the transition table.

Component	Specification
Laptop	Dell Inspiron 15 3525
Processor	AMD Ryzen 5 5625U with Radeon Graphics, 6 cores / 12 threads
Memory	8 GB RAM
Operating system	Windows 11 Home Single Language
Language	Python 3.12 (standard library only, no external libraries)
Web interface	HTML5, CSS3, JavaScript (ES6), served by Python's built-in http.server
Browser	Microsoft Edge (Chromium)
Version control	Git and GitHub

Table 2: System configuration

Algorithm

Building the DFA

1. For every state i from 0 to k and every key x in 0-9:
2. Form the string typed = (first i digits of P) + x; in the accepting state use (P without its first digit) + x, so that the lock keeps watching after opening.
3. Start with j = min(length of typed, k) and decrease j until the last j characters of typed equal the first j digits of P (or j = 0).
4. Set δ(i, x) = j.

Running the lock

1. Start in state 0.
2. For every key pressed, move to δ(state, key) and record the state.
3. If the state equals k, the lock opens.

The naive lock (for comparison)

1. Start with s = 0.
2. If the key equals the next digit of the code, increase s; otherwise set s = 1 if the key equals the first digit, else s = 0.
3. If s = k, open the lock and reset s to 0.

Implementation

The project is written in Python 3 with the standard library only. Table 3 lists its files; the complete source code is included in the submitted ZIP file.

File	What it does
main.py	builds the DFA for a code, prints its transition table, and simulates the DFA lock and the naive lock on key sequences (demonstration, single-sequence and interactive modes)
app.py, index.html	web interface: on-screen keypad, both locks side by side, the state graph and the transition table
api/index.py	the same functions as an online service for the deployed web app
tests/test_lock.py	8 automated unit tests (Chapter IV)

Table 3: Files of the project

Inputs and outputs

Input: a secret code and a sequence of key presses, for example `python main.py --code 1213 --keys 121213`; with `--interactive` the keys are typed live. **Output:** the state path of the DFA and of the naive lock, whether each lock opens, whether it should open, and a table with one row per key. Codes and keys that are not digits are rejected with a clear message.

Making the concept visible

The transition table shows the constructed DFA, and the step-by-step table shows the simulation: for every key the DFA move, the part of the code matched after it, the naive lock's move, and a note where the naive lock forgets a partial match or the DFA opens. The web interface highlights the current state in the state graph (Figures 1 and 2).

Source Code

The listings below are taken directly from the project file main.py.

DFA construction

```
def build_dfa(code):
    """
    Constructs the transition table for the pattern-matching DFA.

    Formal 5-tuple M = (Q, Sigma, delta, q0, F):
      - Q     = {0, 1, ..., k} where k = len(code).
                State i represents: "the longest suffix of keys typed matching
                a prefix of the code has length i".
      - Sigma = {'0', ..., '9'}.
      - delta(state, key) = length of the longest prefix of `code` that is
                also a suffix of (code[:state] + key).
      - q0    = 0 (no digits matched yet).
      - F     = {k} (full code matched -> lock opens).
    """
    k = len(code)
    table = []

    for state in range(k + 1):
        row = {}
        for key in ALPHABET:
```

```
            # If already in the accept state, shift window to continue matching
            if state == k:
                typed = code[1:] + key
            else:
                typed = code[:state] + key

            # Find longest prefix of `code` that matches a suffix of `typed`
            next_state = min(len(typed), k)
            while next_state > 0 and not typed.endswith(code[:next_state]):
                next_state -= 1

            row[key] = next_state
        table.append(row)

    return table
```

Running the DFA and the naive lock

```
def run_dfa(table, code, keys):
    """
    Simulates the DFA on a sequence of key presses.
    Returns: (list of visited states, boolean indicating if the lock opened).
    """
    state = 0
    path = [0]
    opened = False

    for key in keys:
        if key in table[state]:
            state = table[state][key]
        else:
            state = 0
        path.append(state)
        if state == len(code):
            opened = True

    return path, opened

def run_naive(code, keys):
    """
    Simulates a naive greedy pattern matcher.
    On a matching key, it increments state; on a mismatch, it resets to state 0
    (or state 1 if the mismatching key equals code[0]).

    Flaw: It discards overlapping partial matches. For code '1213' and input
    '121213', mismatching on the fourth key resets state to 0 and misses the lock.
    """
    state = 0
    path = [0]
    opened = False
    k = len(code)

    for key in keys:
        if state < k and key == code[state]:
            state += 1
        else:
            state = 1 if key == code[0] else 0

        path.append(state)
        if state == k:
            opened = True
            state = 0

    return path, opened
```

The step-by-step simulation shown to the user

```
def print_steps(code, keys, dfa_path, naive_path, limit=40):
    """Prints every key press: the DFA's move and what it has matched, next to the naive lock's move."""
    if len(keys) > limit:
        print(f"\n  (step-by-step table shown for up to {limit} keys)")
        return
    k = len(code)
    print("\n  Step by step (DFA state i = the last i keys match the first i digits of the code):")
```

```
print(f"  {'step':>4}  {'key':>3}   {'DFA move':<10}{'matched':<10}{'naive move':<12}note")
for i, key in enumerate(keys):
    a, b = dfa_path[i], dfa_path[i + 1]
    na, nb = naive_path[i], naive_path[i + 1]
    note = "DFA opens" if b == k else (f"naive lock forgets '{code[:b]}'" if nb < b else "")
    print(f"  {i + 1:>4}  {key:>3}   {f'{a} -> {b}':<10}{repr(code[:b]):<10}{f'{na} -> {nb}':<12}{
note}")
```

Chapter III - Results

Sample inputs and outputs

Two runs of the program are shown below exactly as it printed them: one key sequence for the code 1213 with the step-by-step table, and the default demonstration, which prints the transition table and compares both locks on six key sequences.

Sample run 1: python main.py --code 1213 --keys 121213

```
============================================================
Evaluation for Code '1213' and Input '121213'
============================================================
  DFA States   : 0 -> 1 -> 2 -> 3 -> 2 -> 3 -> 4
  Naive States : 0 -> 1 -> 2 -> 3 -> 0 -> 1 -> 0
  DFA Lock     : OPEN
  Naive Lock   : SHUT
  Expected     : OPEN
  Result       : Naive lock failed; DFA lock succeeded via suffix fallback.

  Step by step (DFA state i = the last i keys match the first i digits of the code):
  step  key   DFA move  matched   naive move  note
     1    1   0 -> 1    '1'       0 -> 1
     2    2   1 -> 2    '12'      1 -> 2
     3    1   2 -> 3    '121'     2 -> 3
     4    2   3 -> 2    '12'      3 -> 0      naive lock forgets '12'
     5    1   2 -> 3    '121'     0 -> 1      naive lock forgets '121'
     6    3   3 -> 4    '1213'    1 -> 0      DFA opens
```

Sample run 2: python main.py

```
============================================================
Smart Lock DFA: Transition Table for Code 1213
============================================================
  state  meaning          key 1  key 2  key 3  other key
  -----------------------------------------------------
      0  start (0)            1      0      0        0
      1  matched '1'          1      2      0        0
      2  matched '12'         3      0      0        0
      3  matched '121'        1      2      4        0
  [F] 4  matched '1213'       1      0      0        0
  ([F] = accepting state: lock opens)

============================================================
Comparative Evaluation: DFA Lock vs Naive Lock
============================================================
  Keys Pressed    Expected    Naive Lock    DFA Lock    Status
  ------------------------------------------------------------
  1213            opens       opens         opens       Matches expected
  991213          opens       opens         opens       Matches expected
  121213          opens       stays shut    opens       <- Naive lock failed (bug)
  112131213       opens       opens         opens       Matches expected
  1212            stays shut  stays shut    stays shut  Matches expected
  4213            stays shut  stays shut    stays shut  Matches expected

Step-by-step trace for '121213':
  DFA states   : 0 -> 1 -> 2 -> 3 -> 2 -> 3 -> 4
  Naive states : 0 -> 1 -> 2 -> 3 -> 0 -> 1 -> 0
  Explanation  : After typing '1 2 1 2', the fourth key ('2') is a mismatch
  for state 3 ('121'). The naive lock resets completely to state 0.
  The DFA falls back to state 2, recognizing that trailing '1 2' is already
  a valid prefix of the code. The subsequent '1 3' then unlocks the DFA.

To start the web dashboard, run:  python app.py
To test interactive keypad, run: python main.py --interactive
```

In run 1 the fourth key, 2, does not continue '121', but the DFA moves from state 3 to state 2 because '12' is still matched, while the naive lock drops to 0; the table marks the keys where

the naive lock forgets a partial match. The DFA opens on the sixth key; the naive lock never does.

The DFA for the code 1213

State	Meaning	key 1	key 2	key 3	any other key
→ 0	nothing yet	1	0	0	0
1	matched '1'	1	2	0	0
2	matched '12'	3	0	0	0
3	matched '121'	1	2	4	0
4 *	matched '1213'	1	0	0	0

*Table 4: Transition table δ for the code 1213 (→ start state, * accepting state)*

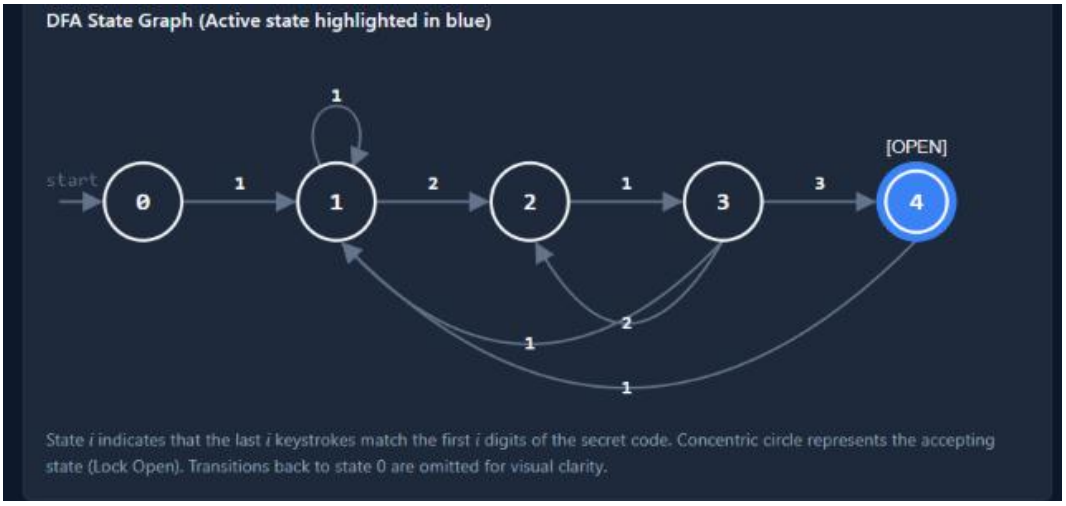


Figure 1: The DFA's state graph in the web interface; the accepting state 4 (OPEN) is highlighted. Transitions back to state 0 are not drawn

DFA lock against naive lock

Keys pressed	Should open?	Naive lock	DFA lock
1213	yes	opens	opens
991213	yes	opens	opens
121213	yes	stays shut	opens
112131213	yes	opens	opens
1212	no	stays shut	stays shut
4213	no	stays shut	stays shut

Table 5: Behaviour of the two locks

On 121213 the DFA visits the states 0 → 1 → 2 → 3 → 2 → 3 → 4 and opens, falling back from state 3 to state 2 on the fourth key. The naive lock visits 0 → 1 → 2 → 3 → 0 → 1 → 0

and stays shut (Figure 2).

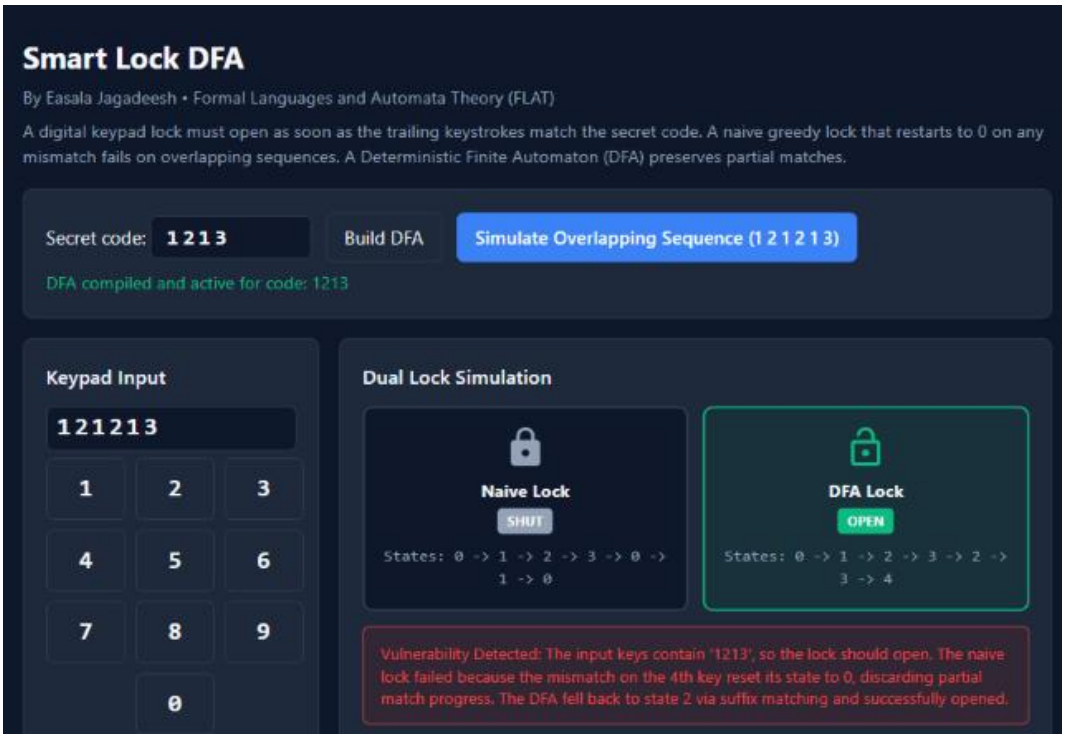


Figure 2: The keys 1 2 1 2 1 3 in the web interface: the naive lock stays shut, the DFA lock opens

How often the naive lock fails

To measure the problem, every sequence of eight key presses over the keys 1-5 (390,625 sequences) was fed to both locks for four codes (Table 6).

Secret code	Sequences that should open	Naive lock missed	DFA lock missed
1213	3,124	74 (2.4%)	0
1122	3,124	434 (13.9%)	0
1111	2,625	0 (0.0%)	0
1234	3,124	0 (0.0%)	0

Table 6: Sequences of 8 keys that contain the code and should open the lock

The naive lock fails whenever a wrong key arrives while the end of the keys typed so far is still the start of the code. This can only happen when the beginning of the code reappears later inside it, as in 1213 (where '1' reappears) and 1122. For codes such as 1234 and 1111 the naive reset happens to be correct, which is why such a bug can easily go unnoticed in testing. The DFA never misses for any code.

Exhaustive verification

The DFA was checked against the definition on every sequence of one to 6 key presses over the keys 1-5 for seven codes (1213, 1111, 1122, 1234, 4545, 121 and 5), 136,710 sequences in total. In every case the DFA opened exactly when the code appeared in the keys.

Chapter IV - Test Cases

Smart Lock DFA was tested on 17 test cases covering the construction of the DFA, the simulation of key sequences, invalid input and very long input. Every 'Actual output' below was produced by running the project code while this report was generated. The unit tests in tests/test_lock.py (run with `python -m unittest discover tests`) check the same behaviour automatically.

Construction of the DFA

No.	Test case	Input	Expected output	Actual output	Result
1	Transition table for 1213	code 1213; columns: keys 1, 2, 3 and any other key (9)	0: 1, 0, 0, 0 | 1: 1, 2, 0, 0 | 2: 3, 0, 0, 0 | 3: 1, 2, 4, 0 | 4: 1, 0, 0, 0	0: 1, 0, 0, 0 | 1: 1, 2, 0, 0 | 2: 3, 0, 0, 0 | 3: 1, 2, 4, 0 | 4: 1, 0, 0, 0	PASS
2	Every move follows the definition	8 codes (1213, 1111, 1122, 1234, 4545, 121, 5, 98765), every state and key	δ = longest prefix that is a suffix in 370 / 370	δ = longest prefix that is a suffix in 370 / 370	PASS
3	Number of states	codes of length k = 1 to 10	k + 1 states for all 10 lengths	k + 1 states for all 10 lengths	PASS

Table 7: Test cases for the construction

Simulation of key sequences

No.	Test case	Input	Expected output	Actual output	Result
4	Code 1213, keys 1213	--code 1213 --keys 1213	OPEN; path 0 1 2 3 4	OPEN; path 0 1 2 3 4	PASS
5	Code 1213, keys 991213	--code 1213 --keys 991213	OPEN; path 0 0 0 1 2 3 4	OPEN; path 0 0 0 1 2 3 4	PASS
6	Code 1213, keys 1212	--code 1213 --keys 1212	SHUT; path 0 1 2 3 2	SHUT; path 0 1 2 3 2	PASS
7	Code 1213, keys (none)	--code 1213 --keys ''	SHUT; path 0	SHUT; path 0	PASS
8	Code 1111, keys 11111	--code 1111 --keys 11111	OPEN; path 0 1 2 3 4 4	OPEN; path 0 1 2 3 4 4	PASS
9	The naive lock's failure	--code 1213 --keys 121213	DFA OPEN (0 1 2 3 2 3 4); naive SHUT	DFA OPEN (0 1 2 3 2 3 4); naive SHUT	PASS
10	Exhaustive check	codes 1213, 1111, 1122; every sequence of 0-7 keys over 1, 2, 3, 9	opens exactly when the code is typed: 65,535 / 65,535	opens exactly when the code is typed: 65,535 / 65,535	PASS

No.	Test case	Input	Expected output	Actual output	Result
11	Naive lock, code with no repeated start	code 1234; all 7-key sequences over 1-4 that contain it	naive lock misses none	naive lock misses none	PASS
12	Naive lock, code whose start repeats	code 1213; all 7-key sequences over 1-4 that contain it	naive lock misses some; DFA none	naive lock misses 8 of 256; DFA none	PASS

Table 8: Test cases for the simulation

Invalid input and performance

No.	Test case	Input	Expected output	Actual output	Result
13	Keys that are not digits	python main.py --keys 12x3	rejected: keys must contain only numeric digits.	rejected: keys must contain only numeric digits.	PASS
14	Code that is not digits	python main.py --code 12a4	rejected: code must contain only numeric digits.	rejected: code must contain only numeric digits.	PASS
15	Empty code	python main.py --code ""	rejected: code must contain only numeric digits.	rejected: code must contain only numeric digits.	PASS
16	Very long input	code 1213; 1,000,002 keys ending in 1213	OPEN in under 2 s	OPEN in 220 ms	PASS
17	Long secret code	a random 30-digit code	31 states, built in under 1 s	31 states, built in 2 ms	PASS

Table 9: Input checking and performance test cases

Analysis of the test results

17 of 17 test cases passed. The results confirm the theory:

- **The construction is correct.** The transition table for 1213 equals the table derived by hand, and in all 370 moves checked for eight codes the DFA move equals its definition, the longest prefix of the code that is a suffix of what was typed. A code of length k always gives k + 1 states.
- **The simulation explains every result.** The state path shows the automaton remembering the longest partial match: after 1212 it is in state 2, and on 121213 it falls back from 3 to 2 instead of restarting, which is exactly why it opens when the naive lock does not.
- **The DFA recognises the language Σ*PΣ* exactly.** On 65,535 key sequences it opened exactly when the code had been typed.

- **The naive design fails only for some codes.** For 1234, whose start never reappears inside it, the naive lock missed nothing; for 1213 it missed 8 of the 256 sequences that should open it. The DFA missed none for either code, so a test with a code like 1234 would hide the bug.
- **Input checking and speed.** Codes and keys that are not digits are rejected with a clear message, and 1,000,002 key presses were processed in 220 ms, one table lookup per key.

Chapter V - Conclusion

This report modelled a keypad lock as a deterministic finite automaton. The language of key sequences that must open the lock is regular, and a DFA for it was constructed systematically: state i records how much of the code is currently matched, and every wrong key leads to the longest match that remains, not back to the start.

The informal design that restarts on every wrong key was shown to miss valid sequences such as 121213, and an exhaustive test of 136,710 sequences confirmed that the DFA has no such failures. The DFA also needs only one table lookup per key and stores just its current state, which makes it well suited to small embedded devices.

The program accepts any code and key sequence as input and shows every move of the automaton next to the naive lock. All 17 of 17 test cases passed, from the hand-derived transition table to invalid input and 1,000,002 key presses.

Future work includes DFA minimisation, a lock that accepts several valid codes (the union of their languages), a limit on the number of wrong attempts, and an implementation on a microcontroller.